PROJECT: Cloud Infrastructure Automation using Terraform and AWS

Services: EC2, RDS, VPC, Terraform

Description: Automate the provisioning of an application stack with Terraform. The stack includes EC2 instances in a VPC with a public and private subnet, a load balancer, and an RDS database.

Skills: Infrastructure as Code (IaC), Terraform, VPC setup

Definition: This project focuses on automating the creation of cloud infrastructure on AWS using Terraform. It provisions resources such as a VPC, public and private subnets, EC2 instances, an Application Load Balancer, and an RDS database. The goal is to manage infrastructure using code instead of manual configuration.

Scope of the Project: The scope of this project includes designing and deploying a scalable and secure cloud environment using Terraform. It covers network setup, compute resources, load balancing, and database deployment within AWS. The project also demonstrates how automation can reduce manual effort and improve consistency in infrastructure management.

Methodologies:

1. **Infrastructure as Code (IaC) using Terraform:** Method used in this Project

- Infrastructure as Code allows cloud infrastructure to be created and managed through code instead of manually configuring resources in the AWS console.
- Terraform automates the provisioning of resources, such as VPCs, subnets, EC2 instances, load balancers, and RDS databases.
- Infrastructure as Code is chosen because it makes the deployment process **automatic, consistent, and repeatable**.
- With Terraform, the entire infrastructure can be created or modified using a single command. It also reduces manual errors and allows easy management and scaling of cloud resources.

2. **Manual configuration using AWS Console – Creating resources step by step in the web interface:** Alternative Method

- Manual configuration means creating each AWS resource step-by-step through the AWS web console.
- This method is not suitable for this project because the project requires automation of infrastructure using Terraform. Manual setup does not provide automation and requires creating each resource individually.

Limitations:

- Time-Consuming
- Difficult to Reproduce Infrastructure
- Hard to Scale

Services Used in this Project:

1. **Amazon EC2 (Elastic Compute Cloud)**

- **Definition:** Amazon EC2 is a cloud service that provides virtual servers to run applications in the AWS environment.
- **Purpose:** EC2 is used to host and run the web application inside the VPC.
- **Role in this Project:** EC2 acts as the application server that receives traffic from the load balancer and processes user requests.

2. **RDS (Relational Database System)**

- **Definition:** Amazon RDS is a managed database service used to create, operate, and scale relational databases in the AWS cloud.
- **Purpose:** RDS is used to store and manage the application's data securely in the private subnet.
- **Role in this Project:** RDS acts as the backend database that stores and retrieves data for the application running on the EC2 instance.

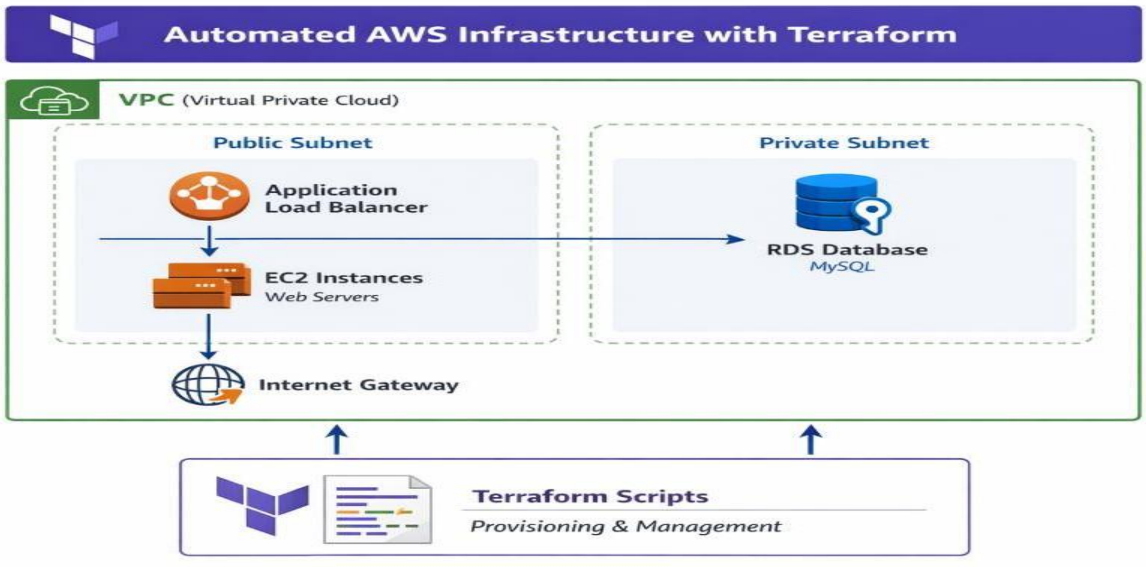
3. **VPC (Virtual Private Cloud)**

- **Definition:** Amazon VPC is a virtual private network in AWS that allows you to securely host and manage cloud resources.
- **Purpose:** VPC is used to create an isolated and secure network environment for deploying the application infrastructure.
- **Role in this Project:** VPC acts as the main network that connects and controls communication between EC2, subnets, load balancer, and RDS.

4. **Terraform**

- **Definition:** Terraform is an Infrastructure as Code (IaC) tool used to create and manage cloud resources using configuration files.
- **Purpose:** Terraform is used to automate the provisioning of AWS infrastructure such as VPC, EC2, load balancer, and RDS.
- **Role in this Project:** Terraform acts as the automation tool that deploys and manages the entire application infrastructure through code.

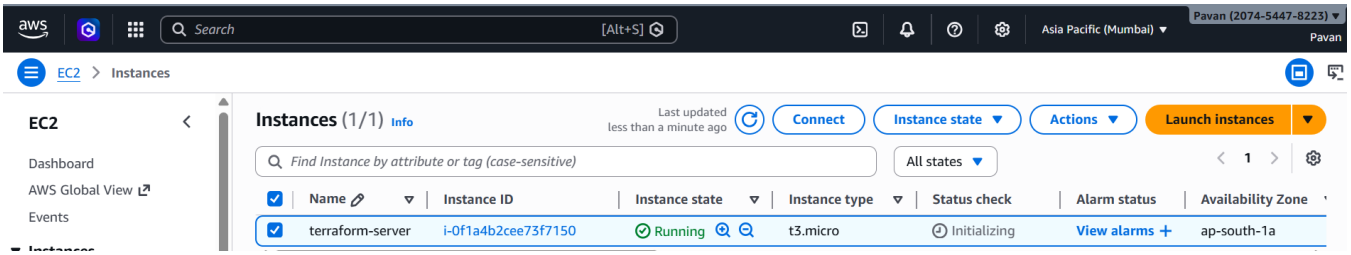
Architecture Diagram



Implementation and Execution of Cloud Infrastructure Automation using Terraform and AWS.

STEP 1: Launch an EC2 Server

- Instance Type: t3. micro
- Volume: 20 GB
- Region: Mumbai (ap-south-1)
- Key Pair: Project-Key
- AMI: Amazon Linux 2023 kernel



STEP 2: Install and Check the Terraform version

```
[ec2-user@ip-10-0-2-168 ~]$ terraform -version
Terraform v1.14.6
on linux_amd64
[ec2-user@ip-10-0-2-168 ~]$
```

STEP 3: Check aws version

```
[ec2-user@ip-10-0-2-168 ~]$ aws --version
aws-cli/2.33.15 Python/3.9.25 Linux/6.1.161-183.298.amzn2023.x86_64 source/x86_64.amzn.2023
[ec2-user@ip-10-0-2-168 ~]$
```

STEP 4: Create Access Keys and Configure AWS CLI

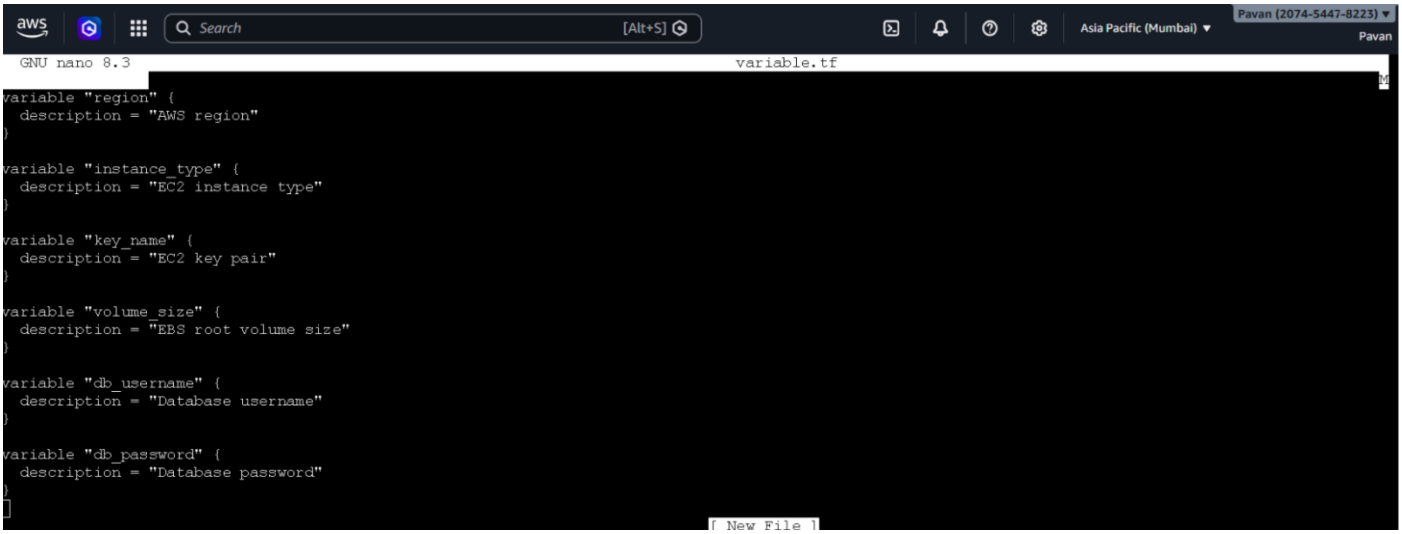
```
[ec2-user@ip-10-0-2-168 ~]$ aws configure
AWS Access Key ID [None]: AKIA5X7FP5TJIY22YXUG
AWS Secret Access Key [None]: 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
Default region name [None]: ap-south-1
Default output format [None]: json
[ec2-user@ip-10-0-2-168 ~]$
```

STEP 5: Create a directory

```
[ec2-user@ip-10-0-2-168 ~]$ mkdir terraform-aws-project
[ec2-user@ip-10-0-2-168 ~]$ ls
terraform-aws-project
```

STEP 6: variable.tf

```
[ec2-user@ip-10-0-2-168 terraform-aws-project]$ nano variable.tf
```



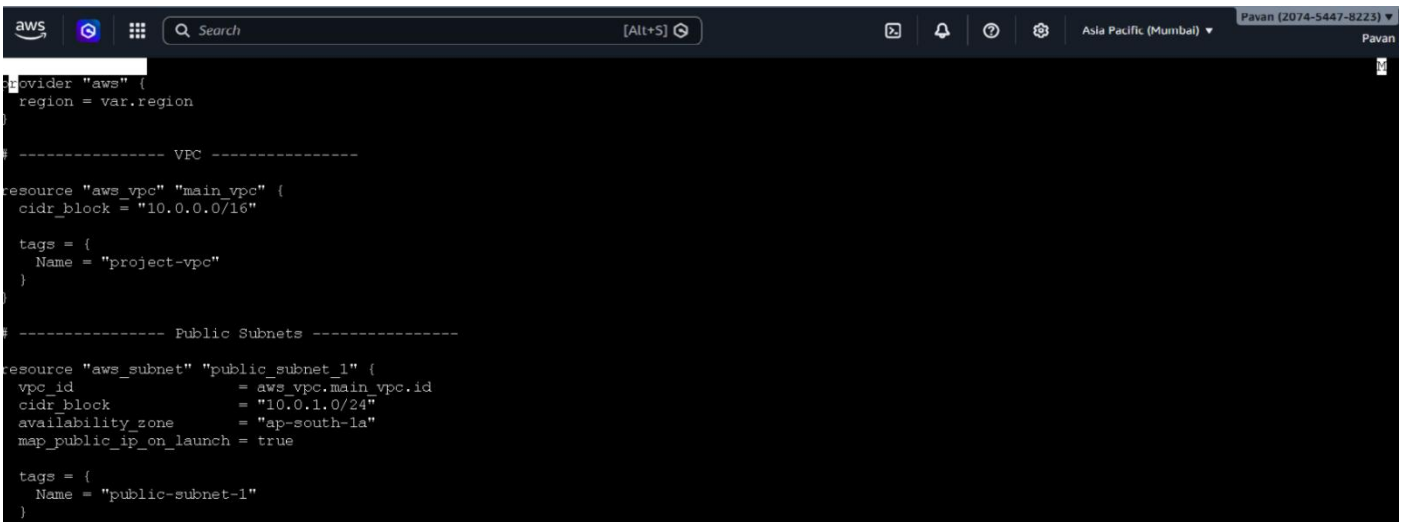
STEP 7: terraform. tfvars

```
[ec2-user@ip-10-0-2-168 terraform-aws-project]$ nano terraform.tfvars
```

STEP 8: output.tf

```
[ec2-user@ip-10-0-2-168 terraform-aws-project]$ nano output.tf
```

STEP 9: main.tf



STEP 10: Created a project directory and added four Terraform files

```
[ec2-user@ip-10-0-2-168 terraform-aws-project]$ nano variable.tf
[ec2-user@ip-10-0-2-168 terraform-aws-project]$ nano main.tf
[ec2-user@ip-10-0-2-168 terraform-aws-project]$ nano terraform.tfvars
[ec2-user@ip-10-0-2-168 terraform-aws-project]$ nano output.tf
[ec2-user@ip-10-0-2-168 terraform-aws-project]$ ls
main.tf  output.tf  terraform.tfvars  variable.tf
[ec2-user@ip-10-0-2-168 terraform-aws-project]$
```

STEP 11: Initialize Terraform

```
[ec2-user@ip-10-0-2-168 terraform-aws-project]$ terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.35.1

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
[ec2-user@ip-10-0-2-168 terraform-aws-project]$
```

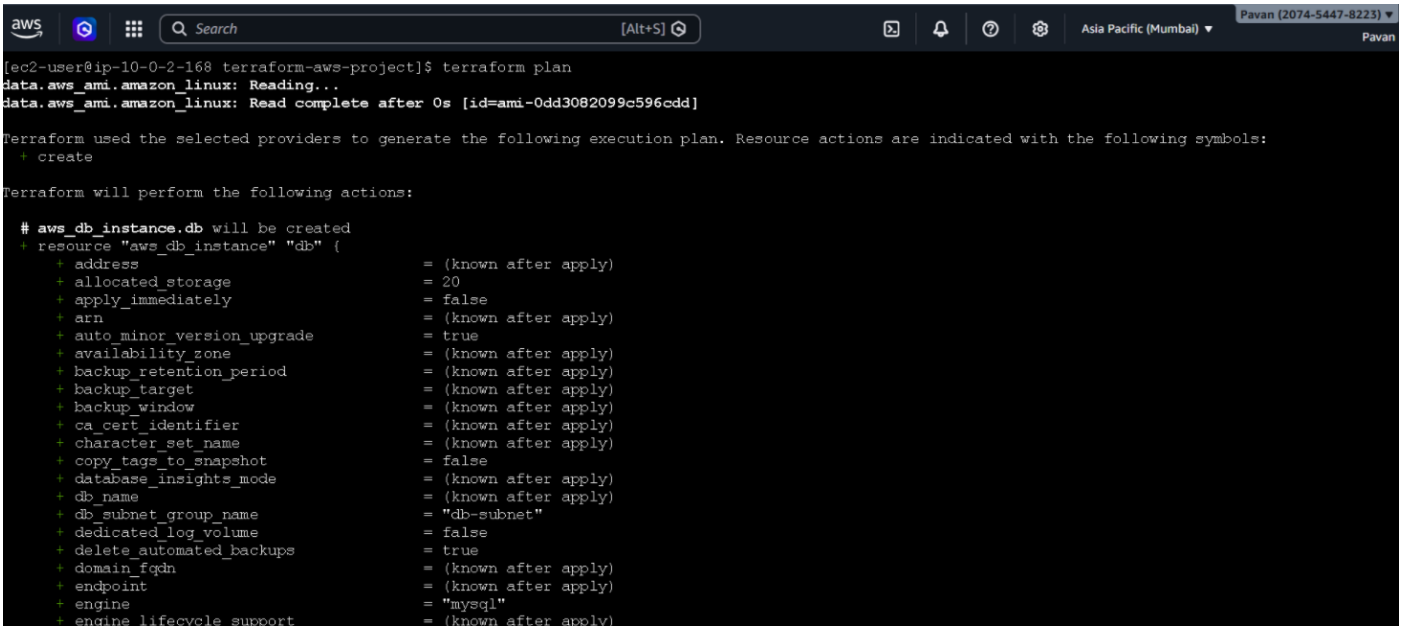
STEP 12: Format Terraform Configuration

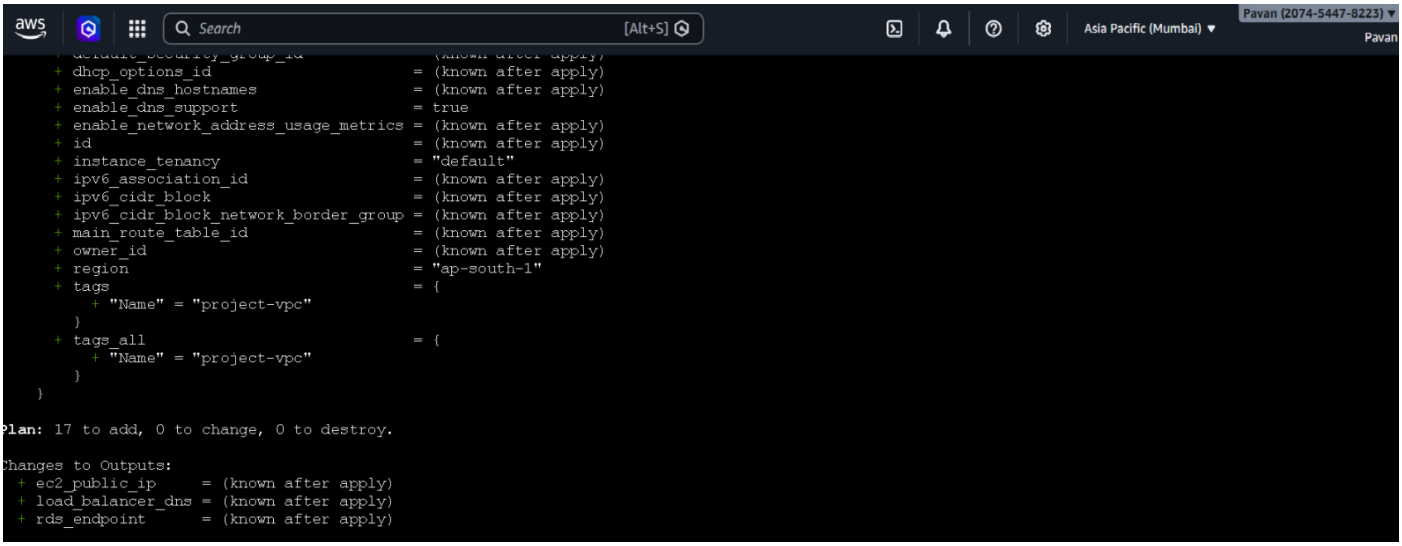
```
[ec2-user@ip-10-0-2-168 terraform-aws-project]$ terraform fmt
main.tf
[ec2-user@ip-10-0-2-168 terraform-aws-project]$
```

STEP 13: Validate Terraform Configuration

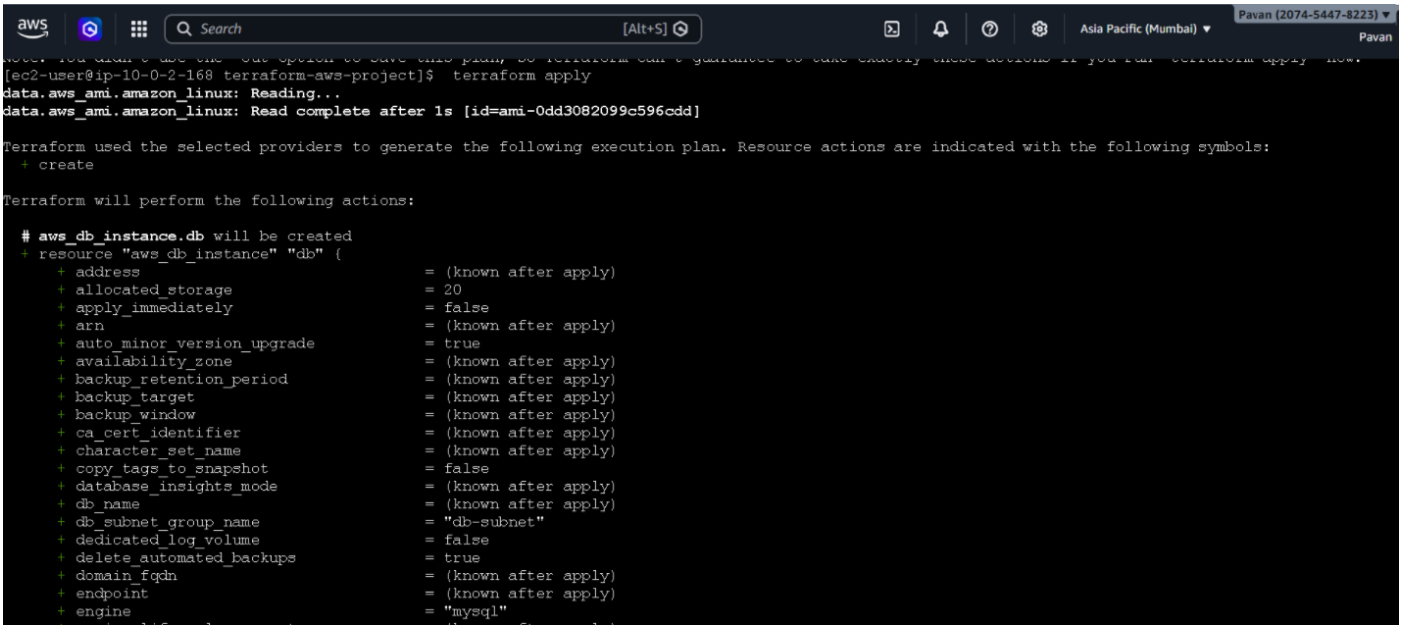
```
[ec2-user@ip-10-0-2-168 terraform-aws-project]$ terraform validate
Success! The configuration is valid.
```

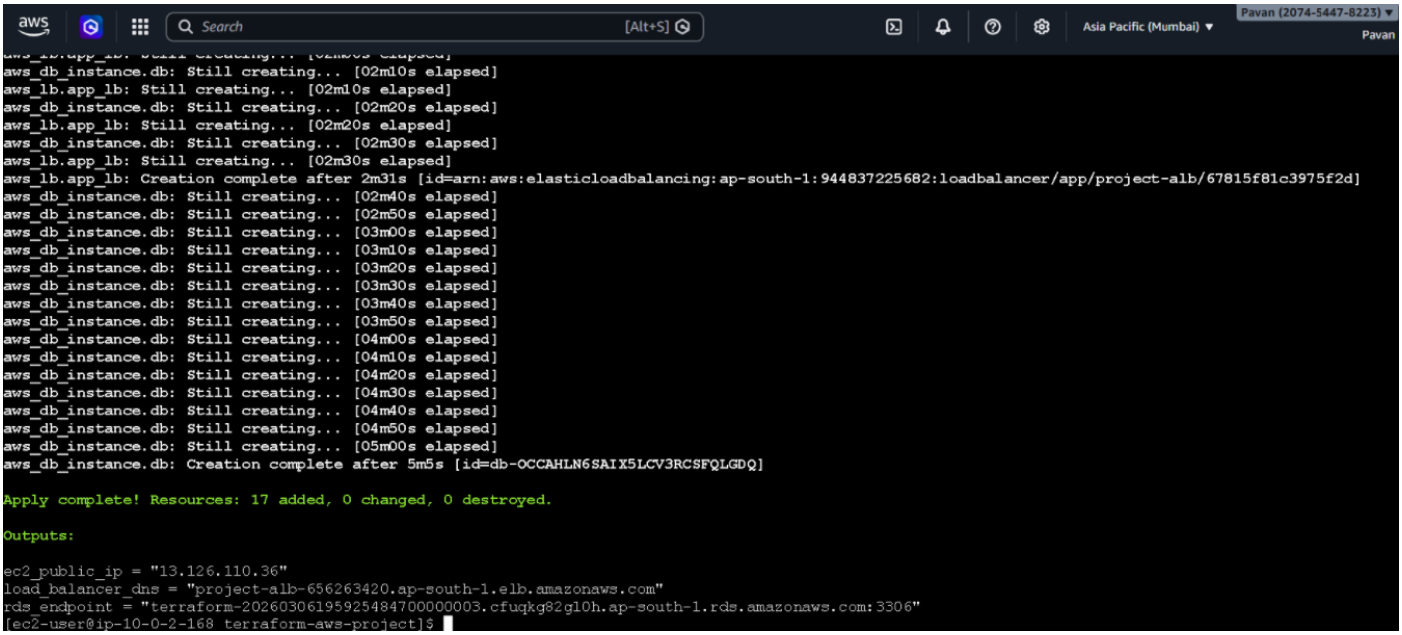
STEP 14: Check Infrastructure Plan



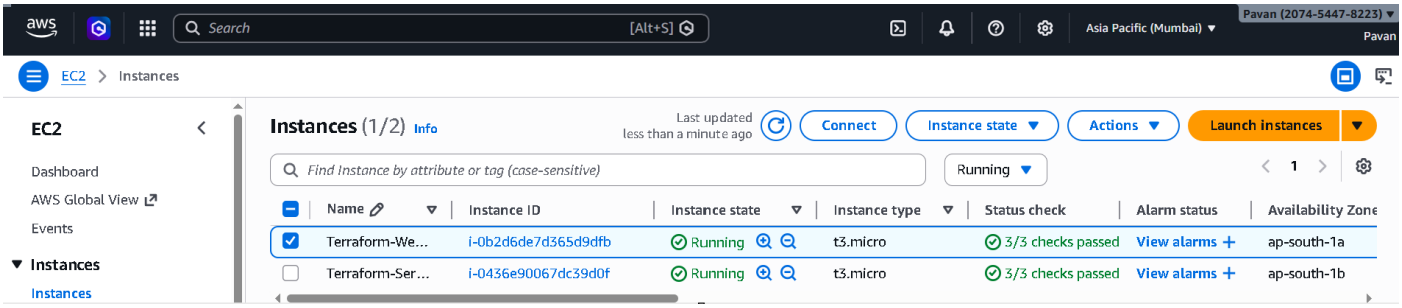


STEP 15: Deploy Infrastructure

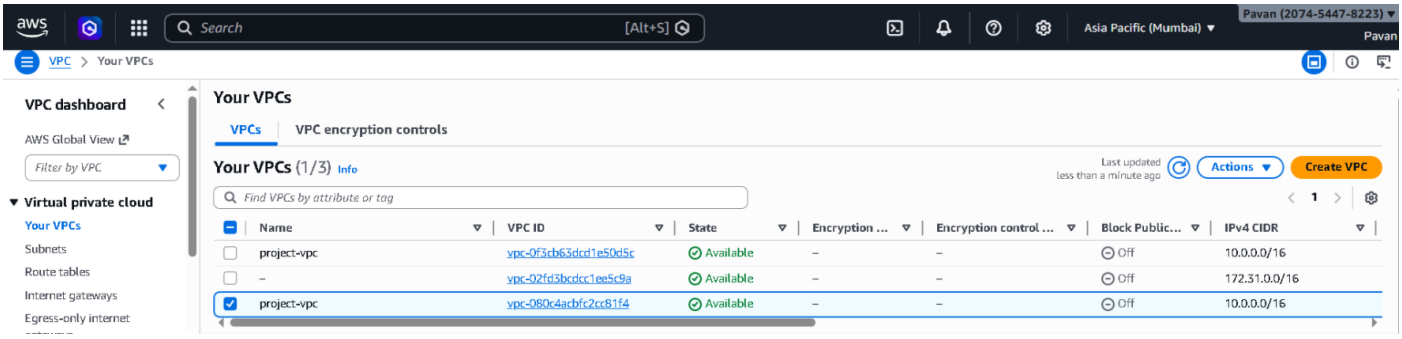




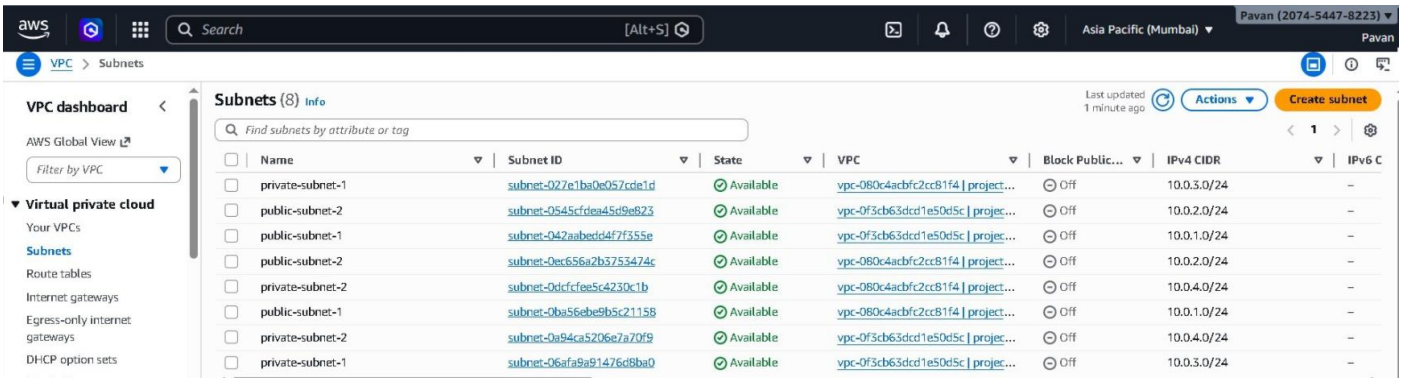
STEP 16: Verify Resources in AWS Console-------→ EC2 (Elastic Compute Cloud)



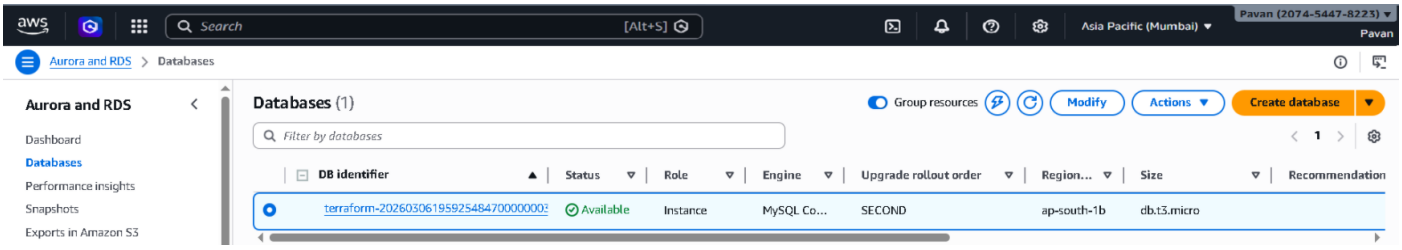
STEP 17: VPC (Virtual Private Cloud)



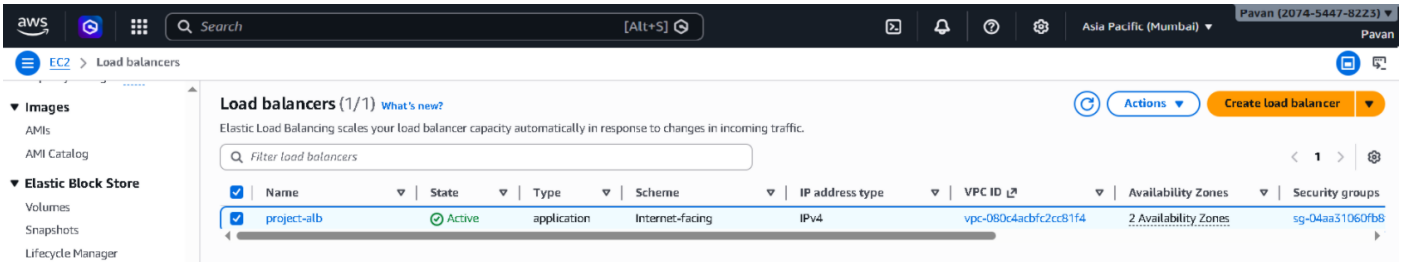
STEP 18: Public Subnet and Private Subnet



STEP 19: RDS (Relational Database Management)



STEP 20: LB (Load Balancer)



Conclusion:

This project successfully demonstrated how to automate the provisioning of cloud infrastructure using Terraform on AWS. By using Infrastructure as Code, resources such as VPC, subnets, EC2 instance, load balancer, and RDS database were created automatically instead of manual configuration. The project helped in understanding AWS networking, compute services, and database deployment. It also showed how automation improves consistency, scalability, and efficiency in cloud infrastructure management. Overall, the project highlights the importance of Terraform in modern DevOps and cloud deployment practices.